

Slide Bài Giảng Seminar T10 — Mutation Testing & Test Effectiveness

Môn học: CS423 / CSC15003 — Kiểm thử phần mềm (FIT@HCMUS)

GVHD: ThS. Hồ Tuấn Thanh

Nhóm 08:

- 23127195 — Trần Mạnh Hùng
- 23127060 — Ninh Văn Khải
- 23127259 — Nguyễn Tấn Thắng

Slide 1: Giới Thiệu Đề Tài

Người phụ trách: Cả Nhóm 08 (23127195 - Trần Mạnh Hùng · 23127060 - Ninh Văn Khải · 23127259 - Nguyễn Tấn Thắng)

Seminar T10: Mutation Testing & Test Effectiveness

Câu hỏi bài học đặt ra: Bạn đã viết test, test đang xanh hết — làm sao biết bộ test đó có thực sự kiểm tra đúng logic không?

Mục tiêu bài giảng:

1. Giải thích tại sao test xanh chưa phải là test tốt.
 2. Giới thiệu kỹ thuật Mutation Testing để đo chất lượng bộ test.
 3. Demo thực tế trên ứng dụng web EShop bằng công cụ Stryker.
-

Slide 2: Vấn Đề Với Bài Test Hiện Tại

Người phụ trách: 23127195 - Trần Mạnh Hùng

Test đang xanh chưa chắc là test đang kiểm tra đúng

Hãy xem ví dụ đơn giản sau.

Hàm kiểm tra người dùng có đủ 18 tuổi không:

```
function isAdult(age) {  
  return age ≥ 18;  
}
```

Bài test được viết:

```
test('kiem tra nguoi lon', () => {  
  expect(isAdult(20)).toBe(true);  
});
```

Kết quả: Test xanh, báo 100% coverage.

Tuy nhiên nếu lập trình viên vô tình đổi $\geq$ thành $>$, bài test đó vẫn xanh. Người 18 tuổi sẽ bị từ chối nhưng không có test nào phát hiện ra.

Vấn đề: Bài test chạy qua dòng code nhưng không kiểm tra logic có đúng không.

Slide 3: Mutation Testing Là Gì?

Người phụ trách: 23127195 - Trần Mạnh Hùng

Ý tưởng: Công cụ tự gieo lỗi nhỏ vào code rồi hỏi bộ test có bắt được không

Cách hiểu đơn giản:

- Bạn viết code và viết test.
- Công cụ Mutation Testing tự động sửa một lỗi nhỏ vào code của bạn.
- Công cụ chạy lại toàn bộ bộ test với code bị sửa đó.
- Nếu bộ test bị FAIL: Bộ test đủ tốt để bắt được lỗi đó.
- Nếu bộ test vẫn PASS: Bộ test chưa kiểm tra được chỗ bị sửa.

Mỗi lần sửa nhỏ như vậy gọi là một Mutant.

Kết quả sau khi chạy test	Ý nghĩa
Bộ test FAIL → Mutant bị diệt	Bộ test đang hoạt động tốt
Bộ test PASS → Mutant sống sót	Bộ test có lỗ hổng, cần cải thiện

Slide 4: Mutant Là Gì? Ví Dụ Cụ Thể

Người phụ trách: 23127195 - Trần Mạnh Hùng

Mutant = một bản copy của code gốc nhưng bị sửa một chỗ nhỏ

Code gốc:

```
if (age ≥ 18) {  
    return 'Duoc phep';  
}
```

Mutant 1 — đổi dấu so sánh từ $\geq$ thành $>$:

```
if (age > 18) {  
    return 'Duoc phep';  
}
```

Mutant 2 — đổi dấu so sánh từ $\geq$ thành $\leq$:

```
if (age ≤ 18) {  
    return 'Duoc phep';  
}
```

Mỗi lần thay đổi như vậy là 1 Mutant. Công cụ có thể tạo ra hàng trăm Mutant từ một file code.

Mục tiêu: Bộ test phải phát hiện và diệt được tất cả các Mutant đó.

Slide 5: Mutation Score Là Gì?

Người phụ trách: 23127195 - Trần Mạnh Hùng

Mutation Score = thước đo chất lượng bộ test

Công thức tính:

$$\text{Mutation Score} = (\text{Số Mutant bị diệt}) / (\text{Tổng số Mutant}) \times 100\%$$

Ví dụ:

- Công cụ tạo ra 100 Mutant.
- Bộ test diệt được 75 Mutant.
- Mutation Score = 75%

Ý nghĩa các mức điểm:

Mức điểm	Đánh giá
Dưới 40%	Bộ test rất yếu
40% - 70%	Bộ test trung bình
70% - 90%	Bộ test tốt
Trên 90%	Bộ test rất chặt chẽ

Lưu ý: Đạt 100% là rất khó vì có những Mutant thay đổi cú pháp nhưng không thay đổi hành vi chương trình.

Slide 6: 4 Trạng Thái Của Một Mutant

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Sau khi chạy bộ test với một Mutant, có 4 kết quả có thể xảy ra

Trạng thái	Biểu tượng	Giải thích
Killed (Diệt)		Bộ test phát hiện được lỗi — bộ test đang hoạt động tốt
Survived (Sống sót)		Bộ test không phát hiện được lỗi — cần viết thêm assertion
Timeout		Mutant gây ra vòng lặp vô tận, test không kết thúc được — tính là diệt
NoCoverage		Không có test case nào chạy đến dòng code bị sửa — thiếu test hoàn toàn

Điểm cần nhớ: Chỉ trạng thái Survived mới là vấn đề thực sự cần xử lý.

Slide 7: Công Cụ Nào Làm Được Việc Đây?

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Có nhiều công cụ Mutation Testing cho từng ngôn ngữ lập trình

Ngôn ngữ	Công cụ phổ biến
JavaScript / TypeScript	Stryker Mutator
Java	PITest
Python	mutmut
C / C++	mull

Nhóm chọn Stryker vì:

- Ứng dụng EShop được viết bằng Node.js và JavaScript.
- Stryker hỗ trợ chạy cùng với Jest — framework viết test phổ biến nhất của JavaScript.
- Stryker xuất báo cáo dạng HTML, hiển thị từng dòng code, tô màu xanh hoặc đỏ theo kết quả.

Slide 8: Bài Test Là Gì? Jest Là Gì?

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Jest là công cụ viết bài kiểm tra tự động cho code JavaScript

Ví dụ một bài test bằng Jest:

```
// Ham can kiem tra:
function tinh_tong(a, b) {
  return a + b;
}

// Bai test:
test('tinh tong hai so', () => {
  const ket_qua = tinh_tong(3, 4);
  expect(ket_qua).toBe(7);
});
```

Giải thích từng dòng:

- `expect(ket_qua)` — Đây là kết quả thực tế hàm trả về.
- `.toBe(7)` — Đây là kết quả kỳ vọng mà mình muốn.
- Nếu hai cái không khớp nhau thì test FAIL.
- Nếu test FAIL khi chạy với Mutant thì Mutant đó bị diệt.

Slide 9: Tại Sao Bộ Test Yếu Không Diệt Được Mutant?

Người phụ trách: 23127060 - Ninh Văn Khải

Test yếu chỉ kiểm tra API có phản hồi không, không kiểm tra nội dung có đúng không

Ví dụ bộ test yếu:

```
test('API đang nhập', async () => {
  const res = await fetch('/api/login', { ... });
  expect(res.status).toBe(200);
  // Chỉ kiểm tra mã trạng thái HTTP, không kiểm tra gì thêm
});
```

Nếu Mutant sửa logic bên trong hàm đăng nhập, API vẫn trả về status 200. Test vẫn PASS. Mutant sống sót.

Cách khắc phục — Thêm assertion kiểm tra chi tiết hơn:

```
test('API đang nhập', async () => {
  const res = await fetch('/api/login', { ... });
  expect(res.status).toBe(200);
  expect(res.body.token).toBeDefined(); // Kiểm tra có token không
  expect(res.body.user.email).toBe('test@mail.com'); // Kiểm tra email có đúng không
});
```

Assertion càng nhiều và càng cụ thể thì Mutant càng dễ bị diệt.

Slide 10: Demo Thực Tế — Chạy Stryker Trên Ứng Dụng EShop

Người phụ trách: 23127060 - Ninh Văn Khải

Stryker hoạt động từng bước như sau

Bước 1: Gõ lệnh chạy Stryker:

```
npm run mutation:auth
```

Bước 2: Stryker tự động thực hiện 4 việc:

1. Đọc file code nguồn `authService.js`.
2. Tạo ra hàng chục Mutant từ file đó.
3. Chạy bộ test auth với từng Mutant một.
4. Ghi lại kết quả: Mutant nào bị diệt, Mutant nào sống sót.

Bước 3: Stryker xuất báo cáo HTML:

- Hiển thị từng dòng code trong trình duyệt.
 - Tô màu xanh cho dòng Mutant bị diệt.
 - Tô màu đỏ cho dòng Mutant sống sót.
 - Người dùng nhấn vào từng dòng đỏ để xem chi tiết cần bổ sung assertion gì.
-

Slide 11: Ví Dụ Thực Tế — Mutant Sống Sốt Và Cách Diệt

Người phụ trách: 23127060 - Ninh Văn Khải

Ví dụ: Stryker đổi điều kiện khóa tài khoản

Code gốc trong hàm đăng nhập:

```
// Kiểm tra tài khoản có đang bị khóa không
if (locked_until && today < locked_until) {
  return { error: 'Tài khoản bị khóa' };
}
```

Mutant do Stryker tạo ra — đổi && thành ||:

```
if (locked_until || today < locked_until) {
  return { error: 'Tài khoản bị khóa' };
}
```

Hậu quả của Mutant: Người dùng bị khóa vĩnh viễn dù thời gian khóa đã hết từ lâu.

Cách diệt Mutant — Viết test case kiểm tra đúng trường hợp này:

```
test('cho phép đăng nhập khi thời gian khóa đã hết', async () => {
  // Đặt thời gian khóa ở 1 giờ trước
  const thờiGianQua = new Date(Date.now() - 3600 * 1000);
  // Cập nhật DB: tài khoản bị khóa từ 1 tiếng trước
  // Thu đăng nhập với mật khẩu đúng, phải được phép
  expect(res.status).toBe(200);
});
```

Slide 12: AI Hỗ Trợ Việc Tìm Và Viết Test Như Thế Nào?

Người phụ trách: 23127060 - Ninh Văn Khải

Quy trình 4 bước: Stryker tìm vấn đề — AI gợi ý — Người kiểm chứng lại

Bước 1: Stryker chạy xong và báo cáo có Mutant sống sót ở dòng 38.

Bước 2: Nhóm đọc báo cáo, thấy Mutant đổi `[user.id]` thành `[]` trong câu lệnh SQL cập nhật database. Nhóm đặt câu hỏi cho AI: Stryker đổi tham số SQL từ `[user.id]` thành mảng rỗng `[]`. Bộ test không phát hiện ra. Gợi ý cách viết test để diệt Mutant này.

Bước 3: AI phân tích code và đề xuất: Sau khi đăng nhập đúng mật khẩu, truy vấn thẳng vào database để kiểm tra cột `login_attempts` có bằng 0 không.

Bước 4: Nhóm tự nhập code gợi ý, chạy `npm test` trên máy để xác nhận test PASS. Sau đó chạy lại Stryker để xác nhận Mutant bị diệt.

Lưu ý quan trọng: AI chỉ gợi ý ý tưởng. Con người phải tự chạy và kiểm chứng lại. AI có thể gợi ý sai.

Slide 13: Hoạt Động Thực Hành — Kill The Mutant (25 Phút)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Cả lớp cùng thử diệt Mutant trực tiếp

Cách chơi:

Bước 1 — 10 phút: Nhóm 08 chiếu 5 Mutant thực tế từ dự án. Mỗi Mutant gồm 3 phần: code gốc, code bị Stryker sửa, và bộ test hiện tại đang không phát hiện được. Nhiệm vụ của mỗi nhóm là viết thêm dòng `expect( ... )` để diệt Mutant đó.

Bước 2 — 5 phút: Các nhóm đổi bài cho nhau chấm chéo.

Bước 3 — 10 phút: Nhóm 08 nhập assertion của từng nhóm rồi chạy `npm test` trực tiếp trên máy chiếu. Nếu test FAIL với Mutant thì nhóm đó thắng.

Slide 14: Tổng Kết — 3 Bài Học Chính

Người phụ trách: Cả Nhóm 08 (23127195 - Trần Mạnh Hùng · 23127060 - Ninh Văn Khải · 23127259 - Nguyễn Tấn Thắng)

Sau bài học hôm nay cần nhớ 3 điều

Điều 1: Test xanh chưa phải là test tốt. Code Coverage chỉ đo test có chạy qua dòng code hay không. Mutation Testing mới đo test có phát hiện lỗi logic hay không.

Điều 2: Assertion càng cụ thể thì test càng tốt. Không nên chỉ viết `expect(status).toBe(200)`. Cần kiểm tra thêm nội dung response body, dữ liệu trả về, trạng thái trong database.

Điều 3: AI hỗ trợ nhanh hơn nhưng con người phải kiểm chứng. AI gợi ý test case tốt nhưng có thể sai. Luôn phải tự chạy lại để xác nhận.

Video Demo (YouTube Unlisted): <<DẤN_YOUTUBE_UNLISTED_LINK_TẠI_ĐÂY>>

Cảm ơn Thầy và các bạn đã theo dõi. Nhóm sẵn sàng nhận câu hỏi.